Trabajo Práctico · Unidad 7

NoSQL y nuevas formas de almacenamiento — Food Store

Tecnicatura Universitaria en Programación · Bases de Datos II · Comisión 4
Integrante: Facundo Chacón · Repositorio: github.com/VIKROOM/DB-TPS-COM4
PostgreSQL 17 (base foodstore) · MongoDB 7.0.14 + mongosh 2.3.1 · 03/11/2026

1. Modelo documental y correspondencia con Food Store (consigna 1)

Se eligió el subconjunto pedidos y detalle de pedidos. En PostgreSQL intervienen pedido (PK id), linea_pedido (PK compuesta pedido_id, producto_id), cliente (PK id, email único) y producto (PK id). Las claves foráneas de linea_pedido referencian al pedido (CASCADE) y al producto (RESTRICT); pedido.cliente_id referencia al cliente (RESTRICT). Cada pedido se representa como un documento con las líneas embebidas:

```
{
  pedido_id: 1,  fecha: ISODate("2026-11-03T01:39:32.658Z"),
  estado: "PENDIENTE",  forma_pago: "EFECTIVO",
  cliente: { cliente_id: 1, nombre: "Ana Gomez" },
  items: [
    { producto_id: 1, nombre: "Muzzarella", categoria: "Pizzas",
      cantidad: 2, precio_unitario: NumberDecimal("1000.00"),
      subtotal: NumberDecimal("2000.00") },
    { producto_id: 2, nombre: "Coca 1.5L", categoria: "Bebidas",
      cantidad: 1, precio_unitario: NumberDecimal("800.00"),
      subtotal: NumberDecimal("800.00") } ],
  items_count: 2,  total: NumberDecimal("2800.00")
}
```

Correspondencia. pedido.id → pedido_id; pedido.fecha → fecha; pedido.forma_pago → forma_pago; cliente.id → cliente.cliente_id; cliente.nombre + apellido → cliente.nombre; linea_pedido + producto → items[] con producto_id, nombre, categoria, cantidad, precio_unitario; SUM(cantidad × precio_unitario) → total. Se conservan los tres identificadores de origen y no se copian email ni teléfono (minimización de datos).

Decisión de agrupamiento. Las líneas se embeben porque siempre se leen juntas, son un arreglo acotado (2 líneas por pedido en la muestra) y permiten actualizar el pedido completo en una escritura atómica. Cliente y producto quedan referenciados por id; solo se copia la instantánea necesaria para mostrar el pedido histórico (nombre, categoría y precio unitario). Es un diseño mixto: referencia estable + copia controlada.

2. Migración y reglas de integridad (consignas 2 y 3)

Se migraron los 12 pedidos con id 1 a 12 y sus 24 líneas, de 11 clientes, con fechas entre 2025-03-01 y 2026-11-02 e importe total de $560.979,00. sql/01_exportar_pedidos_mongodb.sql hace que PostgreSQL genere un insertMany() con ISODate y NumberDecimal; el script mongodb/02_datos_pedidos.js se ejecuta en mongosh. Los documentos incluyen fecha, estado, cliente, productos, cantidades y precios históricos. La colección se crea con $jsonSchema (strict, validationAction: "error") e índices por pedido_id (único), estado, items.producto_id y total.

Regla 1 — identidad y referencia. pedido_id es obligatorio y único; cada items.producto_id debe existir en el catálogo de origen. Verificado: 0 duplicados y 0 líneas huérfanas contra producto.

Regla 2 — coherencia de cantidades e importes. cantidad entero ≥ 1, precio_unitario ≥ 0, subtotal = cantidad × precio_unitario y total = Σ subtotales. Las 24 líneas y los 12 totales coinciden; 23 de 24 precios históricos difieren del precio actual, lo que confirma que la instantánea no se sobrescribe.

Estado. El esquema real no tiene columna de estado. Para la consigna 4.b se deriva con una regla reproducible: fecha_corte = max(fecha del lote) + 1 día; TERMINADO si fecha ≤ fecha_corte − 30 días, PENDIENTE en caso contrario. Se aplica igual en SQL y al migrar; no depende del reloj. Resultado: 11 TERMINADO y 1 PENDIENTE.

3. Consultas y verificación con PostgreSQL (consignas 4 y 5)

#	Consulta MongoDB	Resultado
a	db.pedidos.find({}, {_id:0, pedido_id:1, estado:1}).sort({pedido_id:1})	12 pedidos
b	db.pedidos.find({estado:"TERMINADO"}, {_id:0, pedido_id:1})	11 pedidos: 2–12
c	db.pedidos.find({"items.producto_id": 1}, {_id:0, pedido_id:1})	2 pedidos: 1, 2
d	db.pedidos.find({total: {$gt: NumberDecimal("50000.00")}})	6 pedidos: 2, 6, 8, 10, 11, 12

Umbral de d): $50.000,00. La muestra tiene promedio $46.748,25 y mediana $49.891,50; el umbral separa 6 de 12 pedidos de alto valor, sin ser trivial (no devuelve todos ni ninguno). El importe acumulado es $386.471,00.

Consulta	Cantidad PostgreSQL	Cantidad MongoDB	IDs en ambos motores	¿Coinciden?
b) estado TERMINADO	11	11	2,3,4,5,6,7,8,9,10,11,12	**Sí**
c) producto 1 (Muzzarella)	2	2	1,2	**Sí**

También coinciden los importes: b) $558.179,00 y c) $79.810,00 en ambos motores. No se detectaron diferencias: se migró el mismo subconjunto, se aplicó la misma regla de estado y la misma precisión (NUMERIC(10,2) ↔ Decimal128). La única diferencia es de sintaxis y modelo: SQL usa JOIN y CASE; MongoDB usa notación de punto sobre el arreglo y un campo materializado.

4. Evaluación de la tecnología y contraste de IA (consignas 6 y 7)

Consulta a la IA. Se pidió comparar ambos motores para el modelo actual, el acceso frecuente al pedido completo y la necesidad de conservar coherencia de cantidades y precios (prompt completo en BITACORA_IA.md). La respuesta propuso embeber las líneas y afirmó que MongoDB no soporta transacciones multidocumento, que es más rápido por evitar JOIN, que no admite reglas de integridad y que un documento puede crecer sin límite. Se contrastaron las cuatro afirmaciones:

Afirmación de la IA	Documentación oficial / laboratorio	Veredicto
No hay transacciones ACID multidocumento	Las hay en replica sets; un standalone no las soporta. En el laboratorio: Transaction numbers are only allowed on a replica set member or mongos.	Imprecisa: falsa en general, cierta en nuestro despliegue. El diseño no las necesita: pedido y líneas son un documento.
Es más rápido porque evita JOIN	El embebido reduce operaciones, no garantiza superioridad. Consulta c): MongoDB examinó 2 documentos y 2 claves (1 ms); PostgreSQL usó index-only scans y nested loop (0,086 ms).	No sostenible como absoluto: la ventaja comprobada es de número de operaciones, no de velocidad.
No se pueden imponer reglas de integridad	$jsonSchema con validationAction:"error" rechaza documentos inválidos. El intento con cantidad: 0 devolvió Document failed validation.	Falsa: el esquema flexible admite validaciones estrictas.
El documento crece sin límite	Los documentos BSON tienen un máximo de 16 MiB; el embebido se recomienda para arreglos acotados.	Falsa: los pedidos migrados tienen 2 líneas y ~1,5 KB.

Conclusión (≤ 200 palabras)

Para el subconjunto pedidos-detalle conviene una arquitectura combinada, no un reemplazo. PostgreSQL debe seguir siendo la fuente de verdad: el modelo normalizado garantiza con claves foráneas que cada pedido tenga un cliente y productos existentes, impone cantidades positivas y conserva el precio histórico en linea_pedido; además ofrece transacciones ACID en el despliegue real del equipo. MongoDB aporta una proyección de lectura: un pedido completo se recupera en un solo documento, sin JOIN, y el laboratorio mostró resultados idénticos a SQL (11 pedidos TERMINADO y 2 con el producto 1, mismos IDs e importes). Esa ventaja es de número de operaciones, no una superioridad de rendimiento demostrada con 12 documentos. Los criterios que sostienen la decisión son (1) patrón de acceso —lectura completa y repetida del pedido frente a escrituras transaccionales— y (2) integridad —coherencia de cantidades, precios y referencias—, más (3) costo operativo de mantener dos motores. Si el volumen de lectura lo justifica, sincronizar por pedido_id una colección de solo lectura; migrar el núcleo transaccional no se justifica con la evidencia obtenida.

5. Evidencias, reproducción y límites

Archivos entregados: consultas_mongodb.js y consultas_postgresql.sql (instrucciones ejecutadas); mongodb/01_crear_coleccion.js (validador, índices y metadatos); mongodb/02_datos_pedidos.js (12 documentos generados desde PostgreSQL).

Evidencias: evidencia/mongo_02_carga.txt (12 documentos); mongo_03_verificacion_integridad.txt (9 comprobaciones, todas OK); mongo_04_consultas.txt y postgres_01_consultas.txt (salidas a–d y b–c); mongo_06_pruebas_ia.txt (validador, transacción y explain).

Autoevaluación: verificar_entrega.ps1 ejecuta 26 comprobaciones: archivos exigidos, integridad del modelo, comparación cruzada de IDs e importes entre motores y límite de 5 páginas. Resultado final: 26/26 OK (evidencia/verificacion_entrega.txt).

Límites: el estado es derivado (el esquema relacional no lo tiene); la muestra es pequeña (12 pedidos) y no permite conclusiones de rendimiento; el mongod local es standalone, por lo que no admite transacciones multidocumento. No se afirma que MongoDB reemplace a PostgreSQL.

Bibliografía. MongoDB: Transactions, Schema Validation, Embedded Data y db.collection.find() (documentación oficial). Apunte de la cátedra: «Unidad 7: NoSQL y nuevas formas de almacenamiento».